BACKTRACKING

50 Interview Problems | Java Preparation Guide

Interview-focused A4 study booklet

Includes a priority-wise 50-problem checklist, problem statements, core approaches, complexity guidance, reusable Java templates, and a study checklist.

How to use: Understand the state and choices, write the base case, apply one choice, recurse, and undo the choice. Practise explaining correctness, edge cases, and complexity.

Scope note: Company interviews vary. This is a curated preparation set, not a guarantee of every possible question. Some related problems are more efficiently solved using DP or other methods; these are identified as comparison/related problems.

Priority-wise problem checklist

P1 – Very High

- [ ] Subsets
- [ ] Subsets II
- [ ] Permutations
- [ ] Permutations II
- [ ] Combination Sum
- [ ] Combination Sum II
- [ ] Combination Sum III
- [ ] Letter Combinations of a Phone Number
- [ ] Generate Parentheses
- [ ] Palindrome Partitioning
- [ ] Word Search
- [ ] Restore IP Addresses
- [ ] Combinations
- [ ] Subsequence Sum Equals K
- [ ] Generate Binary Strings Without Consecutive 1s
- [ ] Find All Increasing Subsequences

P2 – High

- [ ] N-Queens
- [ ] N-Queens II
- [ ] Sudoku Solver
- [ ] Rat in a Maze
- [ ] Graph Coloring
- [ ] Hamiltonian Path
- [ ] Hamiltonian Cycle
- [ ] Knight's Tour
- [ ] Word Search II
- [ ] Word Break II
- [ ] Remove Invalid Parentheses
- [ ] Matchsticks to Square
- [ ] Partition to K Equal Sum Subsets
- [ ] Path With Maximum Gold
- [ ] All Paths From Source to Target
- [ ] Beautiful Arrangement
- [ ] Letter Case Permutation
- [ ] Expression Add Operators
- [ ] Crossword Puzzle

P3 – Advanced / Selective

- [ ] Android Unlock Patterns
- [ ] Cryptarithmetic / Verbal Arithmetic
- [ ] Traveling Salesperson – Backtracking
- [ ] Tug of War
- [ ] Maximum Length of a Concatenated String With Unique Characters
- [ ] Split String Into Descending Consecutive Values
- [ ] K-th Permutation
- [ ] All Simple Paths Between Two Vertices
- [ ] Generate Unique Binary Search Trees

- [ ] Boggle Word Search
- [ ] Sudoku with MRV Optimization
- [ ] N-Queens Using Bitmask
- [ ] Assignment Problem – Branch and Bound
- [ ] 0/1 Knapsack – Branch and Bound
- [ ] Meet-in-the-Middle Subset Sum

Problem statements, approaches & complexity

P1 – Very High

Subsets

Problem: Return every subset of an integer array.

Approach: At each index, recurse once without the value and once with it; copy the path at the leaf.

Complexity: O(n*2^n) time; O(n) auxiliary space.

Subsets II

Problem: Return distinct subsets when values repeat.

Approach: Sort; skip equal values at the same recursion depth.

Complexity: O(n*2^n) worst case; O(n) auxiliary.

Permutations

Problem: Generate all orderings of distinct values.

Approach: Choose an unused element, recurse, then unchoose it.

Complexity: O(n*n!) time; O(n) auxiliary.

Permutations II

Problem: Generate unique permutations with duplicates.

Approach: Sort; skip equal values when the previous equal value is not used in this path.

Complexity: O(n*n!) worst case; O(n) auxiliary.

Combination Sum

Problem: Find combinations summing to target; values reusable.

Approach: Recurse from current index and keep the same index after choosing.

Complexity: Exponential/output-sensitive; depth O(target/minValue).

Combination Sum II

Problem: Find unique target combinations; each index used once.

Approach: Sort; recurse with i+1 and skip duplicates at the same depth.

Complexity: O(2^n) worst case; O(n) auxiliary.

Combination Sum III

Problem: Choose k distinct values from 1..9 to sum to n.

Approach: Try increasing values; reduce remaining count and sum.

Complexity: O(C(9,k)*k); O(k) auxiliary.

Letter Combinations of a Phone Number

Problem: Generate letter strings for digits 2–9.

Approach: At each digit, branch over its mapped letters.

Complexity: O(4^n*n); O(n) auxiliary.

Generate Parentheses

Problem: Generate all balanced strings with n pairs.

Approach: Add '(' while open<n; add ')' only while close<open.

Complexity: O(Catalan(n)*n); O(n) auxiliary.

Palindrome Partitioning

Problem: Return all partitions whose pieces are palindromes.

Approach: Try each next substring; recurse only if it is a palindrome.

Complexity: O(n*2^n) upper bound; O(n) auxiliary.

Word Search

Problem: Check whether a word exists in a grid without reusing cells.

Approach: DFS from matching cells; mark visited and restore on return.

Complexity: O(RC*4*3^(L-1)) upper bound; O(L) stack.

Restore IP Addresses

Problem: Insert dots to form valid IPv4 addresses.

Approach: Choose 1–3 digits per segment; reject leading zeros and values >255.

Complexity: Bounded search for input length ≤12.

Combinations

Problem: Choose k values from 1..n.

Approach: Build an increasing path; bound choices so enough values remain.

Complexity: O(C(n,k)*k); O(k) auxiliary.

Subsequence Sum Equals K

Problem: Find subsequences whose sum equals target.

Approach: For each element, branch include/exclude; avoid unsafe pruning with negatives.

Complexity: O(2^n); O(n) auxiliary.

Generate Binary Strings Without Consecutive 1s

Problem: Generate length-n binary strings with no adjacent ones.

Approach: Always branch with 0; branch with 1 only if previous bit is 0.

Complexity: O(F(n)*n) output-sensitive.

Find All Increasing Subsequences

Problem: Return non-decreasing subsequences of length at least two.

Approach: DFS from start index; use a per-depth set to avoid duplicate choices.

Complexity: O(2^n*n) upper bound; O(n) auxiliary.

P2 – High

N-Queens

Problem: Place n queens so none attack each other.

Approach: Place one queen per row; track columns and both diagonals.

Complexity: O(n!) upper bound; O(n) auxiliary.

N-Queens II

Problem: Count N-Queens solutions.

Approach: Use same row-by-row search; increment count at complete placement.

Complexity: O(n!) upper bound; O(n) auxiliary.

Sudoku Solver

Problem: Fill a valid 9×9 Sudoku board.

Approach: Choose an empty cell; try legal digits; undo failed choices. MRV improves search.

Complexity: O(9^E) worst case; O(E) stack.

Rat in a Maze

Problem: Find paths from top-left to bottom-right avoiding blocked cells.

Approach: DFS in allowed directions; mark/unmark current path cells.

Complexity: Exponential; O(RC) auxiliary.

Graph Coloring

Problem: Color graph with at most m colors, adjacent vertices differing.

Approach: Assign a color to a vertex only if no colored neighbor conflicts.

Complexity: O(m^V); O(V) auxiliary.

Hamiltonian Path

Problem: Visit every graph vertex exactly once along edges.

Approach: DFS over adjacent unused vertices; succeed after V vertices.

Complexity: O(V!); O(V) auxiliary.

Hamiltonian Cycle

Problem: Visit each vertex once and return to start.

Approach: Fix a start, DFS unused neighbors, then check closing edge.

Complexity: O(V!); O(V) auxiliary.

Knight's Tour

Problem: Visit every chessboard square exactly once using knight moves.

Approach: Try legal unvisited moves; backtrack; Warnsdorff ordering is a heuristic.

Complexity: O(8^(N²)) upper bound; O(N²) auxiliary.

Word Search II

Problem: Find dictionary words in a board.

Approach: Build a Trie; DFS board paths while following Trie prefixes.

Complexity: Board-dependent exponential; Trie prunes dead prefixes.

Word Break II

Problem: Return all dictionary-based sentence segmentations.

Approach: Try each dictionary word at an index; memoize suffix results.

Complexity: Exponential output; memoized suffix states.

Remove Invalid Parentheses

Problem: Remove minimum parentheses to make string valid; return all results.

Approach: Compute required removals, then DFS with balance pruning and deduplication.

Complexity: O(2^n) upper bound; O(n) auxiliary.

Matchsticks to Square

Problem: Check if sticks form four equal sides.

Approach: Sort descending; place each stick into side buckets; skip symmetric bucket states.

Complexity: O(4^n) upper bound; O(n) auxiliary.

Partition to K Equal Sum Subsets

Problem: Partition array into k equal-sum groups.

Approach: Sort descending; assign to buckets; skip buckets with equal current sums.

Complexity: O(k^n) upper bound; O(n+k) auxiliary.

Path With Maximum Gold

Problem: Collect maximum gold in a non-repeating adjacent path.

Approach: Start DFS at each nonzero cell; mark by zeroing and restore afterward.

Complexity: Exponential in nonzero cells; O(RC) stack.

All Paths From Source to Target

Problem: Return all paths in a DAG from node 0 to node n-1.

Approach: DFS through outgoing edges, append and remove path nodes.

Complexity: Output-sensitive; O(V) stack.

Beautiful Arrangement

Problem: Count permutations where position/value divisibility condition holds.

Approach: At each position, choose an unused valid number.

Complexity: O(n!) upper bound; O(n) auxiliary.

Letter Case Permutation

Problem: Generate strings by toggling letter case.

Approach: For letters branch lower/upper; digits remain unchanged.

Complexity: O(2^L*n); O(n) auxiliary.

Expression Add Operators

Problem: Insert +, -, × between digits to reach target.

Approach: Track evaluated total and last operand to handle multiplication precedence.

Complexity: O(4^n*n) upper bound; O(n) auxiliary.

Crossword Puzzle

Problem: Place given words into slots respecting intersections.

Approach: Choose a slot, try compatible words, recurse, then restore changed cells.

Complexity: Exponential in slots/words.

P3 – Advanced / Selective

Android Unlock Patterns

Problem: Count valid screen patterns respecting intermediate-key rules.

Approach: Precompute midpoint requirements; DFS over unused valid next keys.

Complexity: O(9!); O(9) auxiliary.

Cryptarithmetic / Verbal Arithmetic

Problem: Assign distinct digits to letters to satisfy a word equation.

Approach: Assign digits with no leading zero; stronger column-wise carry search prunes earlier.

Complexity: O(10^L) naive upper bound.

Traveling Salesperson – Backtracking

Problem: Find minimum-cost Hamiltonian cycle.

Approach: DFS over unused cities; prune partial costs no better than best.

Complexity: O(V!); O(V) auxiliary.

Tug of War

Problem: Split array into near-equal-size groups minimizing sum difference.

Approach: Choose exactly floor(n/2) values and track selected sum.

Complexity: O(C(n,floor(n/2))); O(n) auxiliary.

Maximum Length of a Concatenated String With Unique Characters

Problem: Choose strings whose concatenation has no repeated characters.

Approach: Represent character sets as bitmasks; recurse include/exclude.

Complexity: O(2^n*n); O(n) auxiliary.

Split String Into Descending Consecutive Values

Problem: Check if digits split into values decreasing by one.

Approach: Choose first number, then recursively match previous-1; guard overflow.

Complexity: $O(n^2)$ starting choices plus recursive search.

K-th Permutation

Problem: Find k-th lexicographic permutation of 1..n.

Approach: Use factorial number system and remove chosen values; no need to enumerate all permutations.

Complexity: $O(n^2)$ with ArrayList; O(n) space.

All Simple Paths Between Two Vertices

Problem: Enumerate paths without repeating vertices.

Approach: DFS with visited set for current path; unmark on return.

Complexity: Exponential/output-sensitive; O(V) auxiliary.

Generate Unique Binary Search Trees

Problem: Generate all structurally unique BSTs for values 1..n.

Approach: Choose each value as root; combine recursively generated left/right subtrees.

Complexity: Catalan output growth.

Boggle Word Search

Problem: Find dictionary words using neighboring grid letters.

Approach: Trie-guided DFS; clarify whether diagonal moves are allowed.

Complexity: Exponential path search, reduced by Trie.

Sudoku with MRV Optimization

Problem: Solve Sudoku by selecting the empty cell with fewest candidates.

Approach: Maintain candidate sets/masks; branch on most constrained cell.

Complexity: Exponential worst case; heuristic reduces search.

N-Queens Using Bitmask

Problem: Solve/count N-Queens using bit operations.

Approach: Represent columns and diagonals as masks; enumerate available bits.

Complexity: O(n!) upper bound; O(n) recursion.

Assignment Problem – Branch and Bound

Problem: Assign each worker a distinct job at minimum cost.

Approach: DFS assignments; prune using a lower bound for unassigned workers.

Complexity: O(n!) upper bound.

0/1 Knapsack – Branch and Bound

Problem: Maximize value under weight capacity.

Approach: Sort by value/weight; fractional relaxation gives optimistic bound.

Complexity: O(2^n) worst case; O(n) stack.

Meet-in-the-Middle Subset Sum

Problem: Find subset sum by splitting array into halves.

Approach: Enumerate half-sums and search for complementary sums.

Complexity: O(2^(n/2)) time/space; related search optimization.

Java templates for core patterns

1. Include / Exclude (Subsets and subsequence sum)

```
void dfs(int i, int[] a, List<Integer> path, List<List<Integer>> ans) {
    if (i == a.length) {
        ans.add(new ArrayList<>(path));
        return;
    }
    dfs(i + 1, a, path, ans);          // exclude
    path.add(a[i]);                    // choose
    dfs(i + 1, a, path, ans);          // explore
    path.remove(path.size() - 1);      // undo
}
```

2. Start-index loop (combinations)

```
void dfs(int start, int[] a, List<Integer> path) {
    // save path if it satisfies the target condition
    for (int i = start; i < a.length; i++) {
        if (i > start && a[i] == a[i - 1]) continue; // duplicate skip
        path.add(a[i]);
        dfs(i + 1, a, path);
        path.remove(path.size() - 1);
    }
}
```

3. Used array (permutations)

```
void dfs(int[] a, boolean[] used, List<Integer> path) {
    if (path.size() == a.length) {
        // save a copy of path
        return;
    }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        path.add(a[i]);
        dfs(a, used, path);
        path.remove(path.size() - 1);
        used[i] = false;
    }
}
```

4. Grid DFS (word search / maze)

```
boolean dfs(char[][] board, String word, int i, int r, int c) {
    if (i == word.length()) return true;
    if (r < 0 || c < 0 || r == board.length ||
        c == board[0].length || board[r][c] != word.charAt(i))
        return false;
    char old = board[r][c];
    board[r][c] = '#';                 // mark visited
    boolean ok = dfs(board, word, i+1, r+1, c) ||
                 dfs(board, word, i+1, r-1, c) ||
                 dfs(board, word, i+1, r, c+1) ||
                 dfs(board, word, i+1, r, c-1);
    board[r][c] = old;                 // restore
    return ok;
}
```

5. Constraint checking (N-Queens / graph coloring)

```
void dfs(int row) {
    if (row == n) {
        // record/count a complete solution
        return;
    }
    for (int choice = 0; choice < n; choice++) {
        if (!isValid(row, choice)) continue;
        apply(row, choice);
        dfs(row + 1);
        undo(row, choice);
    }
}
```

6. Branch and Bound

```
void dfs(int state, int cost) {
    if (cost >= bestAnswer) return;     // bound-based pruning
    if (isComplete(state)) {
        bestAnswer = Math.min(bestAnswer, cost);
        return;
    }
    for (Choice ch : choices(state)) {
        apply(ch);
        dfs(nextState(state, ch), cost + ch.cost);
        undo(ch);
    }
}
```

Complexity reminder

Backtracking often has exponential worst-case time. State the number of search states/branches, the work per state, recursion depth, and whether output storage is included. Do not quote one universal Big-O for all backtracking problems.

Interview readiness checklist

[ ] Can I define the recursive state precisely?

[ ] Have I identified all choices at each state?

[ ] Is the base case correct?

[ ] Do I validate constraints before recursing?

[ ] Do I restore every modified variable/data structure?

[ ] Can I explain why pruning is safe?

[ ] Can I dry-run a non-trivial example?

[ ] Can I analyse time and auxiliary space?

[ ] Have I tested duplicates, empty input, no-solution cases, and maximum constraints?

[ ] Can I explain when DP or a more efficient algorithm is preferable?

Mastery standard: a problem is complete only when you can solve it without hints, explain the approach, dry-run it, and analyse complexity.